

Backend Server Configuration

The backend of the **DarshanEase – Temple Darshan Ticket Booking System** is developed using **Node.js**, **Express.js**, and **MongoDB**. The backend provides RESTful APIs for user authentication, temple management, darshan slot management, ticket booking, payment processing, and administrative operations.

Server Setup

Create the main backend entry file named **server.js** inside the **server** folder. This file initializes the Express application, connects to the MongoDB database, configures middleware, and starts the server.

Create a **.env** file to store environment variables such as the application port, MongoDB connection string, JWT secret key, and Cloudinary configuration. This enables secure and centralized configuration management.

Example environment variables include:

- PORT
 - MONGO_URI
 - JWT_SECRET
 - CLOUDINARY_CLOUD_NAME
 - CLOUDINARY_API_KEY
 - CLOUDINARY_API_SECRET
-

Configure Express Server

Configure the Express server by installing and using the required middleware such as:

- **CORS** to allow communication between the React frontend and the backend server.
- **Express JSON Middleware** to parse incoming JSON requests.
- **Express URL-Encoded Middleware** to process form data submitted by users.
- **dotenv** to load environment variables.
- **Morgan** (optional) for request logging during development.

These middleware components ensure secure communication and proper handling of client requests.

User Authentication

The DarshanEase system provides secure authentication for devotees, organizers, and administrators.

The authentication module includes:

- User Registration
- User Login
- Password Encryption using **bcrypt.js**
- JWT Token Generation
- User Logout
- Protected API Routes using Authentication Middleware

The authentication middleware verifies JWT tokens before allowing access to secured resources such as booking history, profile management, and administrative operations.

API Route Configuration

Separate route files are created for different modules of the application to maintain a clean and modular project structure.

The backend includes routes for:

- Authentication
- User Management
- Temple Management
- Darshan Slot Management
- Booking Management
- Payment Management
- Admin Management

Each route receives client requests and forwards them to the appropriate controller where the business logic is implemented.

Data Models

The backend uses **Mongoose** to define schemas and interact with the MongoDB database.

The primary data models used in the DarshanEase project include:

- **User Model** – Stores devotee, organizer, and administrator information.
- **Temple Model** – Stores temple details, location, images, and facilities.
- **Darshan Slot Model** – Stores darshan date, timing, seat availability, and pricing.
- **Booking Model** – Stores ticket booking details for each devotee.
- **Payment Model** – Stores payment transaction information and payment status.

These models provide structured storage and efficient database management.

CRUD Operations

The backend supports complete **CRUD (Create, Read, Update, Delete)** functionality for all major modules.

User Module

- Register User
- View User Profile
- Update User Details
- Delete User Account

Temple Module

- Add Temple
- View Temple Details
- Update Temple Information
- Delete Temple

Darshan Slot Module

- Create Darshan Slot
- View Available Slots
- Update Slot Information
- Delete Slot

Booking Module

- Create Booking
- View Booking Details
- Update Booking Status
- Cancel Booking

Payment Module

- Process Payment
- View Payment Details

- Update Payment Status
-

Authentication Middleware

Authentication middleware is implemented to protect secure API endpoints.

Its responsibilities include:

- Verifying JWT Tokens
- Validating User Identity
- Restricting Unauthorized Access
- Allowing Role-Based Access for Users, Organizers, and Administrators

Only authenticated users are allowed to access protected services such as ticket booking, payment processing, booking history, and administrative functions.

Error Handling

A centralized error handling middleware is implemented to improve system reliability and user experience.

The error handler performs the following tasks:

- Captures application exceptions.
- Returns meaningful error messages.
- Sends appropriate HTTP status codes.
- Handles invalid API requests.
- Validates user input.
- Prevents server crashes caused by unexpected errors.

This improves debugging and ensures consistent API responses throughout the application.

Backend Workflow

The backend processes client requests using the following sequence:

Client Request → Express Routes → Controller → Model → MongoDB Database → Controller → API Response → Client

This workflow ensures secure communication between the frontend and backend while maintaining proper separation of concerns through the MVC architecture.